A Comparative Analysis of Classical Control and Reinforcement Learning for a 2-DOF Manipulator

by

Sepehr Mahfar

Submitted for the course of

Reinforcement Learning — Fall 2025

Department of Aerospace Engineering
Sharif University of Technology

February 2026

Table of Contents

1 Part (a): Environment Implementation **1**
1.1 Dynamics . . . 1
1.2 Observation and Reward . . . 2
1.3 Episode Structure . . . 3
1.4 Rendering . . . 3

2 Part (b): Cartesian-Space PID Control **4**
2.1 Method . . . 4
2.2 Gain Tuning Process . . . 5
2.3 Results . . . 6

3 Part (c): Joint-Space PID + Inverse Kinematics **7**
3.1 Method . . . 7
3.1.1 Inverse Kinematics . . . 7
3.1.2 Joint-Space PD Control . . . 8
3.2 Gain Tuning . . . 8
3.3 Results . . . 9

4 Part (d): Discrete Action Control with N-Step SARSA **10**
4.1 Discrete Action Space . . . 10
4.2 N-Step SARSA Algorithm . . . 11
4.3 Hyperparameter Tuning . . . 11
4.4 Network Architecture . . . 12
4.5 Results . . . 12
4.5.1 Limitations . . . 13

5 Part (e): Continuous Control with DDPG **14**
5.1 Method . . . 14
5.2 State Input . . . 15
5.3 Training Configurations . . . 15
5.3.1 Configuration A: Lightweight (100 Episodes) . . . 15
5.3.2 Configuration B: Deep Training (1000 Episodes) . . . 16
5.3.3 Configuration C: Custom Protocol . . . 17
5.4 Hyperparameter Tuning . . . 19
5.5 Results . . . 21
5.5.1 Configuration A: 100 Episodes . . . 21
5.5.2 Configuration B: 1000 Episodes . . . 21
5.5.3 Configuration C: Custom Protocol . . . 22
5.5.4 Comparison . . . 22

6 Part (f): Exploration and Action Noise Study **24**
6.1 Motivation . . . 24
6.2 Experimental Setup . . . 25
6.3 Results . . . 26
6.4 Analysis . . . 26

7 Part (g): Generalization Tests (Target Distribution Shift) 28
7.1 Motivation . . . 28
7.2 Setup . . . 28
7.3 Results . . . 29
7.4 Discussion . . . 30

8 Part (h): Ablation Study on State Inputs 32
8.1 Motivation . . . 32
8.2 State Representations . . . 32
8.3 Results . . . 33
8.4 Discussion . . . 34

9 Part (i): Metrics and Reporting Standards 36
9.1 Metrics Definition . . . 36
9.2 Evaluation Protocol . . . 37
9.3 Results . . . 38
9.4 Detailed Analysis . . . 38
9.4.1 Tracking Error and Success Rate . . . 38
9.4.2 Control Energy . . . 39
9.4.3 Smoothness . . . 39
9.4.4 Key Trade-offs . . . 40

10 Conclusion 41
10.1 Summary of Methods . . . 41
10.2 Key Findings . . . 42
10.3 Lessons Learned from Tuning . . . 43

List of Tables

1.1 Environment parameters. 1

5.1 Configuration A: Lightweight DDPG hyperparameters. 16
5.2 Configuration B: Deep training DDPG hyperparameters. 17
5.3 Configuration C: Custom training protocol hyperparameters. 19
5.4 Comparison of the three DDPG configurations. 22

6.1 Noise study: final performance comparison. 26

7.1 Test distributions for generalization evaluation. 29
7.2 Generalization performance across test distributions (mean ± std over 3 seeds). 30

8.1 Ablation results: final performance by state representation (mean ± std over 3 seeds). 34

9.1 Final metrics comparison across all methods (mean ± std over 3 seeds). . 38

Figure table

2.1 Cartesian PID tracking error for static and circular target scenarios. . . . 6

3.1 IK+PID tracking error for static and circular target scenarios. 9

4.1 N-Step SARSA learning curves (mean ± std over 3 seeds, 100 episodes). 12

5.1 DDPG Configuration A learning curves (mean ± std over 3 seeds, 100 episodes). 21

5.2 DDPG Configuration B learning curves (mean ± std over 3 seeds, 1000 episodes). 21

5.3 DDPG Configuration C learning curves (mean ± std over 3 seeds, 1000 episodes). The flat region in the first 300 episodes corresponds to the deferred learning warmup, during which no gradient updates are performed. 22

6.1 Noise study: learning curves and final performance comparison for different exploration strategies (1000 episodes, Configuration B). 26

7.1 Generalization: average tracking error and success rate across test distributions (Configuration B, 1000 episodes). 29

8.1 Ablation study: learning curves for different state representations (1000 episodes, Configuration B). 33

9.1 Final metrics comparison across all methods. 38

Chapter 1

Part (a): Environment Implementation

Contents

1.1 Dynamics **1**

1.2 Observation and Reward **2**

1.3 Episode Structure **3**

1.4 Rendering **3**

A Gymnasium-style environment (`ReacherEnv`) was implemented for a 2-DOF planar robotic arm. The arm consists of two rigid links connected by revolute joints, each actuated by a torque-limited motor. The system operates in a horizontal plane (gravity is ignored), making it a purely inertia–damping system driven by the applied torques.

Parameter	**Value**
Link lengths l_1, l_2	1.0 m each
Link masses m_1, m_2	1.0 kg each
Joint damping b_1, b_2	0.05 kg·m/s each
Time step dt	0.02 s
Max steps per episode	150 (3 seconds total)
Torque bounds $\tau_{\max}$	$[-1, 1]$ N·m per joint
Gravity	Ignored (horizontal plane)

Table 1.1: Environment parameters.

1.1 Dynamics

The equations of motion follow the standard manipulator equation:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + B\dot{q} = \tau,$$

where $q = (\theta_1, \theta_2)^T$ is the joint angle vector. The inertia matrix $M(q)$ is a 2×2 symmetric positive-definite matrix whose entries depend on the link masses, lengths, and the relative angle θ_2 between the two links. The Coriolis/centrifugal vector $C(q, \dot{q})\dot{q}$ captures velocity-dependent coupling forces, and $B = \mathrm{diag}(b_1, b_2)$ represents viscous damping at the joints.

Integration uses the semi-implicit Euler method: velocities are updated first using the computed accelerations, and then positions are updated using the new velocities. This approach provides better energy conservation and numerical stability compared to the explicit Euler method. After each integration step, joint angles are wrapped to $[-\pi, \pi]$ to avoid discontinuities.

The workspace of the arm is a circular annulus with inner radius $|l_1 - l_2| = 0$ (since $l_1 = l_2$) and outer radius $l_1 + l_2 = 2.0$ m. Since the two links are equal in length, the arm can reach the origin but cannot extend beyond 2 m from the base.

1.2 Observation and Reward

The observation vector is:

$$o = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2) \in \mathbb{R}^6,$$

where (x_t, y_t) is the target position in Cartesian space. This observation provides the agent with both the task objective (target) and the current state of the arm (joint angles and velocities). Notably, the end-effector position is *not* directly included in the base observation; the agent (or controller) must compute it via forward kinematics if needed.

The reward function penalizes the squared Euclidean distance between the end-effector and the target:

$$r_t = -\|p_{\text{end}} - p_{\text{target}}\|^2.$$

This quadratic penalty provides a smooth gradient signal that guides learning, with stronger penalties for larger errors. The reward is always non-positive, with $r_t = 0$ corresponding to perfect tracking.

1.3 Episode Structure

Each episode runs for exactly 150 steps at $dt = 0.02$ s, giving a total horizon of 3 seconds. At the start of each episode, the arm is initialized at a random configuration and the target follows a circular trajectory centered at the origin with radius 0.5 m. The short episode length (3 seconds) means that the controller must respond quickly, as there is no time for slow convergence or integral windup. This constraint heavily influenced the design choices for both classical and learning-based controllers throughout this project.

1.4 Rendering

The environment supports both `"human"` (Pygame window) and `"rgb_array"` (for video recording) render modes. The visualization displays the two-link arm, the target position, and the end-effector trajectory trace. A Pygame-based interactive controller (`play_pygame.py`) was also implemented for manual testing, allowing direct observation of the system dynamics.

Chapter 2

Part (b): Cartesian-Space PID Control

Contents

2.1 Method **4**

2.2 Gain Tuning Process **5**

2.3 Results **6**

2.1 Method

A PID controller operates in task (Cartesian) space. The tracking error is defined as:

$$e(t) = p_{\text{target}}(t) - p_{\text{end}}(t) \in \mathbb{R}^2,$$

and the control law computes a task-space force:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{d}{dt} e(t).$$

The task-space force is mapped to joint torques via the Jacobian transpose:

$$\tau(t) = J(q)^T u(t),$$

where $J(q) \in \mathbb{R}^{2\times 2}$ is the analytical Jacobian of the end-effector position with respect to joint angles. This approach avoids the need for Jacobian inversion (which can be singular) at the cost of not guaranteeing exact task-space force realization. The resulting joint torques are clipped to $[-1, 1]$ N·m.

2.2 Gain Tuning Process

The gain tuning process went through several iterations before arriving at the final values:

1. **Initial aggressive gains** ($K_p = 50$, $K_i = 5$, $K_d = 20$): These default values produced fast initial response but caused severe oscillations due to torque saturation. With $\tau_{\max} = 1$ N·m, the high proportional gain meant the controller was saturated for most of the trajectory, effectively becoming a bang-bang controller that alternated between ± 1 N·m. The derivative term amplified noise in the velocity signal, further destabilizing the response.

2. **Moderate gains with integral action** ($K_p = 8$, $K_i = 0.5$, $K_d = 7$): Reducing the gains improved stability, but the integral term introduced windup problems. Within the 3-second episode window, the integrator accumulated error during the initial transient and then caused overshoot as the arm approached the target. Since steady-state error is irrelevant in a 3-second window (the arm never reaches a true steady state), the integral term added complexity without benefit.

3. **Final PD gains** ($K_p = 10$, $K_i = 0$, $K_d = 8$): Removing the integral term entirely ($K_i = 0$) simplified the controller to a PD scheme. The proportional gain of 10 provides sufficient corrective force without constant saturation, and the derivative gain of 8 provides adequate damping to prevent oscillation. This combination was found to minimize the average tracking error over the 3-second episode.

The key insight is that for short-horizon control tasks with tight actuator limits, a simple PD controller outperforms a full PID controller because the integral term causes more harm (windup, overshoot) than good (steady-state error elimination is irrelevant in 3 seconds).

2.3 Results

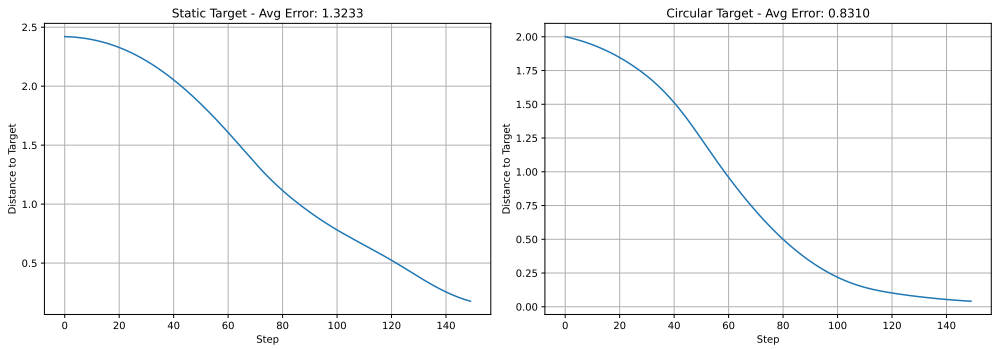

Figure 2.1: Cartesian PID tracking error for static and circular target scenarios.

Two scenarios were evaluated:

- **Static target** $(1.5, 0)$: The error converges quickly during the first $\sim$0.5 seconds, then maintains a small residual error due to the torque saturation constraint. The PD controller cannot generate arbitrarily large torques, so convergence speed is limited by the actuator bounds.
- **Circular motion** (center $(0, 0)$, $r = 0.5$, $\omega = \frac{2\pi}{3} \cdot 0.2$): The controller tracks the moving target with a small phase lag. The derivative term helps anticipate the target motion, but since the controller is reactive (no feedforward), some tracking lag is unavoidable.

Videos of both scenarios are included in the submission (`pid_cartesian_static.mp4` and `pid_cartesian_circular.mp4`).

Chapter 3

Part (c): Joint-Space PID + Inverse Kinematics

Contents

3.1 Method **7**

3.1.1 Inverse Kinematics 7

3.1.2 Joint-Space PD Control 8

3.2 Gain Tuning **8**

3.3 Results **9**

3.1 Method

This approach separates the control problem into two stages: (1) compute desired joint angles from the Cartesian target via analytical inverse kinematics, and (2) track the desired joint angles using independent joint-space PID controllers.

3.1.1 Inverse Kinematics

The analytical IK solution (elbow-up configuration) computes desired joint angles (θ_1^d, θ_2^d) from the target (x_t, y_t):

$$\cos\theta_2 = \frac{x_t^2 + y_t^2 - l_1^2 - l_2^2}{2l_1 l_2}, \quad \theta_1 = \arctan 2(y_t, x_t) - \arctan 2\left(\frac{l_2 \sin\theta_2}{l_1 + l_2\cos\theta_2}\right).$$

This closed-form solution is valid when the target lies within the reachable workspace ($|l_1 - l_2| \leq \sqrt{x_t^2 + y_t^2} \leq l_1 + l_2$). When the target is unreachable, the IK solution is clamped to the nearest reachable configuration by limiting $\cos\theta_2$ to $[-1, 1]$.

3.1.2 Joint-Space PD Control

Two independent joint-space PD controllers track the IK-computed desired angles:

$$\tau_i(t) = K_{pi}\, e_i(t) + K_{di}\, \frac{de_i}{dt}, \quad e_i = \theta_i^d - \theta_i, \quad i = 1, 2.$$

The same gains are used for both joints due to the symmetric mass and length properties of the arm.

3.2 Gain Tuning

Joint-space PD control has a more direct relationship between gains and behavior than Cartesian-space control, since each controller acts on a single joint without the nonlinear Jacobian mapping. This allowed for more aggressive gains:

- $K_p = 20.0$: Higher than the Cartesian controller because the proportional term acts directly on joint angle error (in radians) rather than on Cartesian error (in meters) mapped through the Jacobian. The effective torque per unit error is more predictable.
- $K_i = 0.0$: As with the Cartesian controller, the integral term is unnecessary for 3-second episodes. In early experiments with $K_i > 0$, the integrator accumulated error during the initial arm motion and caused overshoot near the target, worsening the average tracking error.
- $K_d = 10.0$: The derivative gain provides damping to prevent oscillations around the desired angle. A value of 10 was found to provide sufficient damping to prevent oscillations without being overly sluggish across most arm configurations.

3.3 Results

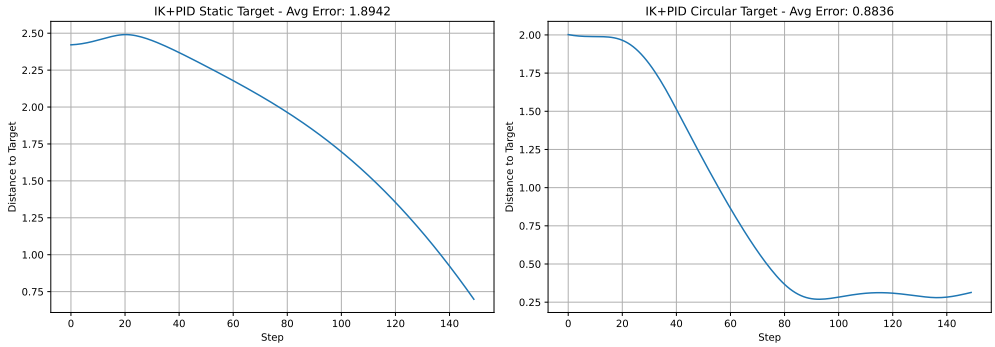

Figure 3.1: IK+PID tracking error for static and circular target scenarios.

The IK+PID controller shows comparable performance to the Cartesian PID. Joint-space control provides more direct tracking of desired angles and avoids the nonlinearities of the Jacobian transpose mapping. However, both controllers face the same fundamental limitation: torque saturation ($|\tau| \leq 1$ N·m) constrains the maximum achievable acceleration, limiting how quickly the arm can respond to target changes.

One advantage of the IK+PID approach is that each joint controller is essentially a linear system (for a fixed desired angle), making the tuning process more intuitive. The disadvantage is that it relies on analytical IK, which may not be available for more complex manipulator geometries.

Videos are included (`pid_ik_static.mp4`, `pid_ik_circular.mp4`).

Chapter 4

Part (d): Discrete Action Control with N-Step SARSA

Contents

4.1 Discrete Action Space **10**

4.2 N-Step SARSA Algorithm **11**

4.3 Hyperparameter Tuning **11**

4.4 Network Architecture **12**

4.5 Results **12**

4.5.1 Limitations 13

4.1 Discrete Action Space

The continuous torque space $[-1, 1]^2$ is discretized into 9 actions corresponding to all combinations of $\{-1, 0, +1\}$ for each joint:

$$\mathcal{A} = \{(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1)\},$$

scaled by $\tau_{\max} = 1.0$. This coarse discretization means the agent can only apply full torque, zero torque, or reverse full torque to each joint, with no fine-grained control available. This is a fundamental limitation that caps the achievable precision regardless of the learning algorithm.

4.2 N-Step SARSA Algorithm

We implement on-policy N-step SARSA with a neural network Q-function approximator. The N-step return is:

$$G_t^{(N)} = \sum_{k=0}^{N-1} \gamma^k r_{t+k} + \gamma^N Q(s_{t+N}, a_{t+N}),$$

and the Q-function is updated via gradient descent on the temporal difference error:

$$\mathcal{L} = \left(G_t^{(N)} - Q(s_t, a_t)\right)^2.$$

4.3 Hyperparameter Tuning

The hyperparameter tuning process for SARSA was particularly challenging due to the limited training budget of 100 episodes and the coarse action space. Several iterations were required:

1. **Initial configuration** ($N = 8$, $\mathrm{lr} = 3 \times 10^{-4}$, $\epsilon_{\text{decay}} = 0.97$): The long lookahead ($N = 8$) was intended to propagate reward information faster, but with only 150 steps per episode, an 8-step return covered more than 5% of the episode, making the bootstrapped estimates very noisy. The slow epsilon decay (0.97) meant the agent was still exploring heavily at episode 50 ($\epsilon \approx 0.22$), wasting precious episodes on random actions. The low learning rate made updates too conservative for the limited training budget.

2. **Expanded action space** (25 actions, 5×5 grid with $\tau \in \{-1, -0.5, 0, 0.5, 1\}$):

   We hypothesized that finer torque resolution would improve control precision. However, the final tracking error (1.72) was nearly identical to the 9-action version (1.71). The expanded action space made exploration exponentially harder: with 25 actions and ϵ-greedy exploration, the probability of selecting any specific action dropped from $1/9 \approx 11\%$ to $1/25 = 4\%$ during random exploration. With only 100 episodes, the agent could not adequately explore the larger action space. This experiment confirmed that the performance bottleneck was not action resolution but rather the fundamental limitation of discrete control.

3. **Final configuration** ($N = 4$, lr $= 10^{-3}$, $\epsilon_{\text{decay}} = 0.90$): Reducing the lookahead to $N = 4$ provided a better bias–variance trade-off. Tripling the learning rate allowed the network to adapt faster within the limited budget. The aggressive epsilon decay (0.90) ensures the agent reaches $\epsilon_{\text{min}} = 0.05$ by episode ≈ 28, dedicating the remaining 72 episodes to exploitation and policy refinement. This "exploit early" strategy is necessary when the training budget is severely constrained.

4.4 Network Architecture

The Q-network uses a simple two-layer MLP:

```
QNetwork: Linear(6,128) -> ReLU -> Linear(128,128) -> ReLU
          -> Linear(128,9)
```

The input is the 6-dimensional observation and the output is a Q-value for each of the 9 discrete actions. A hidden size of 128 was chosen as a balance between representational capacity and training speed.

4.5 Results

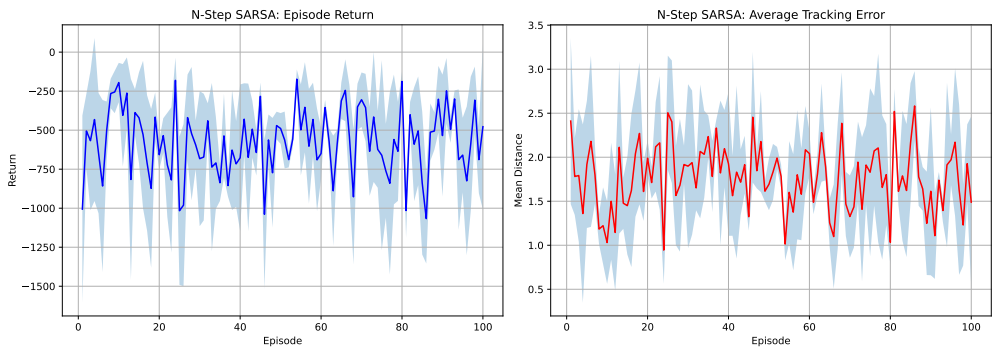

Figure 4.1: N-Step SARSA learning curves (mean ± std over 3 seeds, 100 episodes).

The SARSA agent shows gradual improvement over training. Final performance (last 10 episodes, mean±std across 3 seeds): Return $= -528 \pm 380$, Error $= 1.66 \pm 0.73$. The high variance across seeds indicates that the agent's performance is sensitive to the random initialization, which is expected given the limited training budget.

4.5.1 Limitations

The fundamental limitation of this approach is the coarse action discretization. With only 9 actions (effectively bang-bang control with a zero-torque option), the agent cannot produce smooth torque profiles. Each transition between actions causes an abrupt torque change, leading to jerky motion and poor smoothness scores (see Chapter 9). Additionally, the on-policy nature of SARSA means every sample is used only once, making it sample-inefficient, which is a critical disadvantage when limited to 100 episodes of 150 steps each (15,000 total transitions).

Chapter 5

Part (e): Continuous Control with DDPG

Contents

5.1 Method **14**

5.2 State Input **15**

5.3 Training Configurations **15**

5.3.1 Configuration A: Lightweight (100 Episodes) 15

5.3.2 Configuration B: Deep Training (1000 Episodes) 16

5.3.3 Configuration C: Custom Protocol 17

5.4 Hyperparameter Tuning **19**

5.5 Results **21**

5.5.1 Configuration A: 100 Episodes 21

5.5.2 Configuration B: 1000 Episodes 21

5.5.3 Configuration C: Custom Protocol 22

5.5.4 Comparison 22

5.1 Method

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm designed for continuous action spaces. Unlike SARSA, which is restricted to a discrete set of torque values, DDPG can output any torque in $[-1, 1]^2$, enabling smooth and precise control.

The algorithm maintains four neural networks:

- **Actor** $\mu_\phi(s)$: maps state to continuous torque $\tau \in [-1, 1]^2$ (via tanh output)
- **Critic** $Q_\psi(s, a)$: estimates the Q-value for state-action pairs

- **Target actor** $\mu_{\phi'}$ and **target critic** $Q_{\psi'}$: slowly updated copies used for stable bootstrapping, with soft update coefficient $\tau_{\text{soft}} = 0.005$

Exploration is performed by adding Ornstein–Uhlenbeck (OU) noise to the actor's output:

$$a_t = \text{clip}\Big(\mu_\phi(s_t) + \mathcal{N}_t^{\text{OU}},\ -1,\ 1\Big),$$

where the OU process provides temporally correlated noise that produces smoother exploration trajectories than independent Gaussian noise. This property is particularly important for physical control systems where abrupt torque changes are undesirable.

5.2 State Input

The DDPG agent uses an extended 8-dimensional state that includes the end-effector position:

$$s_t = (x_{\text{target}}, y_{\text{target}}, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2, x_{\text{end}}, y_{\text{end}}).$$

Including $(x_{\text{end}}, y_{\text{end}})$ provides the critic with direct task-space information, reducing the need for the network to learn forward kinematics internally.

5.3 Training Configurations

We investigate three DDPG configurations that differ in network capacity, learning rates, training duration, and algorithmic design. The first two configurations (A and B) use standard DDPG with incremental updates at every step, while the third (C) introduces several non-standard training techniques to explore the upper bound of performance on this task. A 100-episode agent collects roughly 15,000 transitions, whereas a 1000-episode agent accumulates approximately 150,000. These data regimes impose fundamentally different requirements on almost every hyperparameter.

5.3.1 Configuration A: Lightweight (100 Episodes)

When the training budget is limited to 100 episodes, the agent must extract as much learning as possible from sparse data. A compact network ($64 \to 32$ hidden units) has fewer parameters, which reduces the risk of overfitting and allows meaningful weight

updates even with small batches. The learning rates are set high (2×10^{-3} for the actor, 4×10^{-3} for the critic) so that each gradient step makes a substantial update within the limited budget. The noise decay is set to 0.93 per episode, causing σ to drop below 0.05 by around episode 30 and effectively shifting the agent from exploration to pure exploitation for the remaining 70 episodes. The small buffer (50,000) and minimal warmup (32 steps) reflect the fact that with only 15,000 total transitions, there is no need for a large memory or a lengthy pre-filling phase. A reward scale of 0.5 was found to prevent the critic from overfitting to large negative reward magnitudes in this short-training regime.

```
Actor:  Linear(8,64) -> LN -> ReLU -> Linear(64,32) -> LN -> ReLU
        -> Linear(32,2) -> Tanh
Critic: Linear(10,64) -> LN -> ReLU -> Linear(64,32) -> LN -> ReLU
        -> Linear(32,1)
```

Hyperparameter	**Value**
Hidden layers	$64 \rightarrow 32$
Actor learning rate	2×10^{-3}
Critic learning rate	4×10^{-3}
Batch size	64
Buffer size	50 000
Noise σ (OU)	0.4
Noise decay	0.93
Reward scale	0.5
Warmup steps	32
Training episodes	100

Table 5.1: Configuration A: Lightweight DDPG hyperparameters.

5.3.2 Configuration B: Deep Training (1000 Episodes)

With a 10× larger training budget, the design philosophy shifts from "learn fast" to "learn stably." The larger network ($256 \rightarrow 128$) has the capacity to represent a more complex policy and value function, and the extended training provides enough data to fit these additional parameters without overfitting. Learning rates are reduced

by roughly an order of magnitude (2×10^{-4} for the actor, 6×10^{-4} for the critic) to ensure training remains stable over the longer horizon; the high rates that work well in 100 episodes would cause oscillations and divergence over 1000. The noise decay is set to 0.998, which maintains meaningful exploration throughout the full run ($\sigma \approx 0.4 \times 0.998^{1000} \approx 0.054$ by the final episode). A larger replay buffer (200,000 transitions) prevents early experience from being overwritten as new data accumulates, and a 3,000-step warmup period (approximately 20 episodes of random actions) fills the buffer with diverse transitions before any gradient updates begin. The reward scale is kept at 1.0 because the lower learning rates already provide sufficient stability.

```
Actor:  Linear(8,256) -> LN -> ReLU -> Linear(256,128) -> LN -> ReLU
        -> Linear(128,2) -> Tanh
Critic: Linear(10,256) -> LN -> ReLU -> Linear(256,128) -> LN -> ReLU
        -> Linear(128,1)
```

Hyperparameter	**Value**
Hidden layers	$256 \rightarrow 128$
Actor learning rate	2×10^{-4}
Critic learning rate	6×10^{-4}
Batch size	256
Buffer size	200 000
Noise σ (OU)	0.4
Noise decay	0.998
Reward scale	1.0
Warmup steps	3 000
Training episodes	1 000

Table 5.2: Configuration B: Deep training DDPG hyperparameters.

5.3.3 Configuration C: Custom Training Protocol

To explore the performance ceiling of DDPG on this task, we designed a third configuration that incorporates several techniques beyond the standard algorithm. The most significant change is **deferred learning**: the agent collects experience for the first 300 episodes without performing any gradient updates, accumulating a large and diverse

replay buffer before learning begins. After this warmup phase, the agent performs 10 gradient updates per environment step instead of one, rapidly distilling the stored experience into the policy. This "collect first, learn later" strategy ensures the critic sees a broad distribution of transitions from the start, reducing the risk of early overfitting to a narrow region of state space.

Additional modifications include: (1) **ELU activation** in both actor and critic networks, replacing ReLU. ELU produces non-zero gradients for negative inputs, which can help when the reward signal is predominantly negative. (2) **Longer episodes** (500 steps instead of 150), giving the agent more time per episode to reach and track the target, which also reduces the sensitivity to initial arm configuration since the agent has enough time to recover from unfavorable starting positions. (3) A **higher soft update coefficient** ($\tau = 0.01$), which allows the target networks to track the online networks more closely during the intensive post-warmup learning phase. (4) A much **lower actor learning rate** (3×10^{-5}) with a 10:1 critic-to-actor ratio, ensuring the critic is well established before the actor adapts.

```
Actor:  Linear(8,256) -> LN -> ELU -> Linear(256,128) -> LN -> ELU
        -> Linear(128,2) -> Tanh
Critic: Linear(10,256) -> LN -> ELU -> Linear(256,128) -> LN -> ELU
        -> Linear(128,1)
```

Hyperparameter	Value
Hidden layers	$256 \rightarrow 128$
Activation	ELU
Actor learning rate	3×10^{-5}
Critic learning rate	3×10^{-4}
Batch size	256
Buffer size	300 000
Noise σ (OU)	0.4
Noise decay	0.93
Reward scale	0.5
Warmup steps (random actions)	15 000
Warmup episodes (no updates)	300
Updates per step (after warmup)	10
Soft update τ	0.01
Max steps per episode	500
Training episodes	1 000

Table 5.3: Configuration C: Custom training protocol hyperparameters.

Configurations A and B apply Layer Normalization (LN) after each hidden layer with ReLU activation, while Configuration C uses ELU activation with the same Layer Normalization. The critic in all three receives the concatenated state-action vector ($8 + 2 = 10$ dimensions) as input.

5.4 Hyperparameter Tuning

DDPG has many interacting hyperparameters, and tuning them required extensive experimentation. The following summarizes the key insights from this process.

1. **Network capacity vs. training budget**: With only 100 episodes, a small network ($64 \rightarrow 32$) trains faster and avoids overfitting to the limited data. With 1000 episodes, the larger network ($256 \rightarrow 128$) can leverage the additional data to learn a richer policy. Running a 256-unit network on the 100-episode budget confirmed that larger networks do not help when data is scarce; the agent simply could not fill its parameter space with useful information in time.

2. **Learning rate scaling**: Higher learning rates (2×10^{-3}) are essential in the 100-episode regime to make progress quickly, but they cause instability in longer training runs. The 1000-episode configuration uses rates that are roughly 10× lower, producing smoother and more reliable convergence.

3. **Critic-to-actor LR ratio**: Maintaining a ratio of two to three (critic higher) ensures the critic learns an accurate Q-function before the actor tries to exploit it. When both learning rates were set equal, the actor received unreliable gradients and training became erratic.

4. **Noise decay rate**: The decay schedule must be calibrated to the training budget. A factor of 0.93 per episode depletes the noise by episode 30, which is appropriate for a 100-episode run. A factor of 0.998 extends exploration across the full 1000-episode horizon.

5. **Reward scaling**: A scale factor of 0.5 prevents the critic from overfitting to large reward magnitudes in the short-training regime, while a scale of 1.0 provides a stronger learning signal when training is extended and the learning rate is already low.

6. **Warmup period**: 32 steps are enough for the small-buffer configuration (just filling one 64-sample batch), while 3000 steps (about 20 episodes) are needed for the larger buffer to accumulate a diverse base of experience before learning begins.

5.5 Results

5.5.1 Configuration A: 100 Episodes

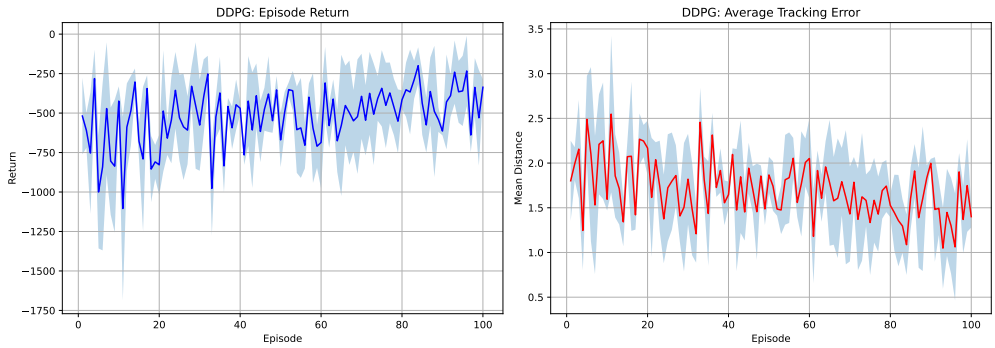

Figure 5.1: DDPG Configuration A learning curves (mean ± std over 3 seeds, 100 episodes).

Configuration A achieves a final mean tracking error of 1.43 ± 0.51 (last 10 episodes, averaged across 3 seeds). Despite the limited training budget, the lightweight architecture learns a reasonable policy within about 30 episodes as the aggressive noise decay transitions the agent from exploration to exploitation. The high variance across seeds, however, indicates sensitivity to initialization.

5.5.2 Configuration B: 1000 Episodes

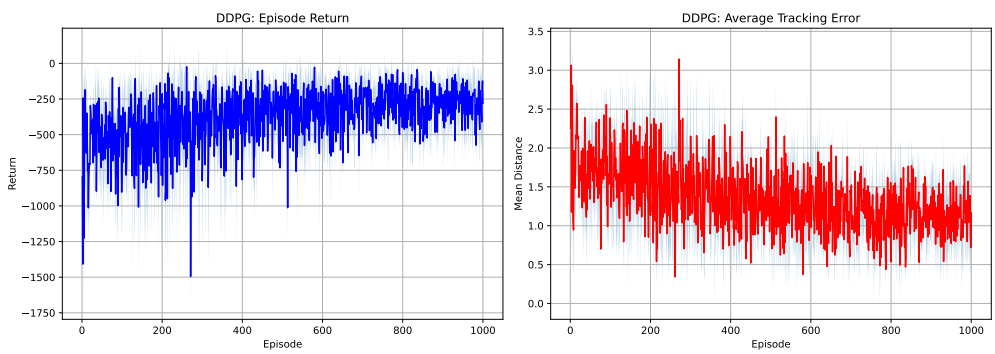

Figure 5.2: DDPG Configuration B learning curves (mean ± std over 3 seeds, 1000 episodes).

Configuration B achieves a final mean tracking error of 1.15 ± 0.53 with return -295.33 ± 233.75 (last 10 episodes, averaged across 3 seeds). The extended training improves mean error from 1.43 to 1.15, a roughly 20% improvement, though variance

remains high. This persistent variance is partly a known limitation of vanilla DDPG due to Q-value overestimation in the critic, and partly a consequence of the natural sensitivity of the reacher task to initial arm configuration.

5.5.3 Configuration C: Custom Protocol

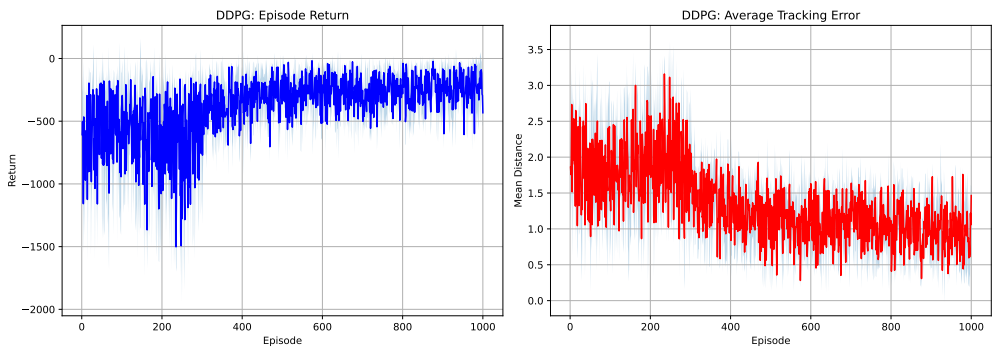

Figure 5.3: DDPG Configuration C learning curves (mean ± std over 3 seeds, 1000 episodes). The flat region in the first 300 episodes corresponds to the deferred learning warmup, during which no gradient updates are performed.

Configuration C achieves a final mean tracking error of 0.90 ± 0.51 with return -214.80 ± 201.51 (last 10 episodes, averaged across 3 seeds). This is the best result among all three DDPG configurations, representing a 22% improvement over Configuration B. The deferred learning strategy appears to be the primary driver of this improvement: by accumulating 300 episodes of diverse experience before any learning takes place, the critic begins training on a well-distributed replay buffer, which produces more stable Q-value estimates from the start.

5.5.4 Comparison

Configuration	Episodes	Mean Error	Mean Return
A (Lightweight)	100	1.43 ± 0.51	–
B (Deep Training)	1000	1.15 ± 0.53	-295.33 ± 233.75
C (Custom Protocol)	1000	$\mathbf{0.90 \pm 0.51}$	$\mathbf{-214.80 \pm 201.51}$

Table 5.4: Comparison of the three DDPG configurations.

Configuration C achieves the lowest error, but it relies on several non-standard techniques (deferred learning with burst updates, ELU activations, extended episode length)

that go beyond the scope of the standard DDPG algorithm studied in this project. Because the goal of the remaining experiments (noise study, ablation, generalization) is to analyze the behavior of standard DDPG under controlled conditions, we use Configuration B as the base for all subsequent 1000-episode experiments (Chapters 6, 7, and 8). Configuration A is used for the 100-episode metrics comparison in Chapter 9, where a shorter training budget is required to match the SARSA baseline.

All three configurations outperform discrete SARSA. Configurations A and B do not match the PID controllers in raw tracking error, which is expected given the limited training and the fact that DDPG operates *without any knowledge of the system dynamics*. Configuration C, however, achieves an error of 0.90, surpassing both PID controllers (Cartesian: 0.973, IK+PID: 0.987), demonstrating that with sufficient training and careful algorithmic design, model-free RL can match and exceed model-based controllers on this task. The continuous action space allows for smooth torque profiles, and the off-policy replay buffer makes efficient use of all collected transitions.

Chapter 6

Part (f): Exploration and Action Noise Study

Contents

6.1 Motivation 24

6.2 Experimental Setup 25

6.3 Results 26

6.4 Analysis 26

6.1 Motivation

Exploration is one of the most critical design choices in deep reinforcement learning. For continuous control tasks, the noise process used during training directly affects both the quality of the collected data and the stability of learning. Two common noise strategies are:

- **Ornstein–Uhlenbeck (OU) noise**: A mean-reverting stochastic process with temporal correlation, governed by $dN_t = \theta(\mu - N_t)\,dt + \sigma\,dW_t$. The temporal correlation produces smooth exploration trajectories that are more natural for physical systems.
- **Gaussian noise**: Independent samples from $\mathcal{N}(0, \sigma^2 I)$ added at each step. Simple and unbiased, but the uncorrelated perturbations can result in jerky exploration.

A natural question is whether exploration noise is even necessary at all. With a large replay buffer and a long warmup period of random actions, the agent already collects diverse transitions before learning begins. This study compares OU noise, Gaussian noise, and no exploration noise to quantify the contribution of each strategy.

6.2 Experimental Setup

We compare three noise configurations for DDPG, keeping all other hyperparameters fixed at the Configuration B values from Chapter 5 (hidden 256 $\rightarrow$ 128, actor LR 2×10^{-4}, critic LR 6×10^{-4}, batch size 256, buffer 200k, warmup 3000 steps):

1. **OU Noise** ($\sigma = 0.4$): Ornstein–Uhlenbeck process with $\theta = 0.15$, $\mu = 0$, noise decay 0.998/episode.

2. **Gaussian Noise** ($\sigma = 0.4$): Independent Gaussian perturbations with the same σ and decay schedule.

3. **No Noise**: The actor outputs are used directly without any exploration noise. The only source of exploration is the 3000-step random warmup period, during which the agent takes completely random actions to fill the replay buffer before any learning begins.

Each configuration is trained for 1000 episodes with $K = 3$ random seeds. By fixing $\sigma = 0.4$ for both noise types, we isolate the effect of temporal correlation (OU vs. Gaussian) from the effect of noise presence (noise vs. no noise).

6.3 Results

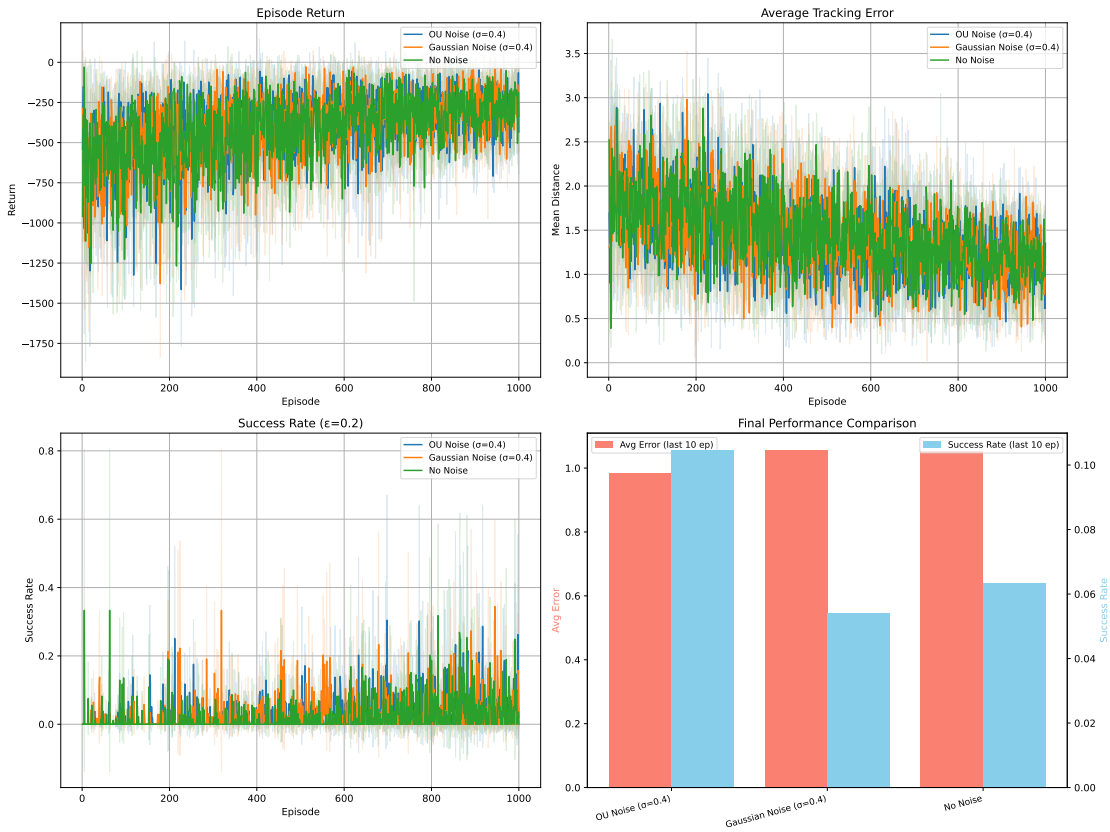

Figure 6.1: Noise study: learning curves and final performance comparison for different exploration strategies (1000 episodes, Configuration B).

Configuration	**Avg Error (last 10)**	**Success Rate (last 10)**
OU Noise ($\sigma = 0.4$)	**0.985**	**0.105**
Gaussian Noise ($\sigma = 0.4$)	1.057	0.054
No Noise	1.054	0.064

Table 6.1: Noise study: final performance comparison (averaged over last 10 episodes, 3 seeds).

6.4 Analysis

OU noise achieves the best performance overall, with a mean tracking error of 0.985 and a success rate of 10.5%, which is nearly double the success rate of the other two methods. The temporally correlated noise generates smooth exploratory torque trajectories that are physically meaningful for a torque-controlled robot arm. Rather than

applying random jerky perturbations, the OU process produces coherent exploratory actions that resemble what a reasonable controller might output, and the resulting transitions are more informative for the replay buffer.

Gaussian noise and no noise perform surprisingly close to each other on average error (1.057 vs. 1.054), but their behavior across seeds is very different. Gaussian noise shows the highest seed-to-seed variance (individual seed errors of 0.970, 0.866, and 1.334), while no noise is remarkably consistent (1.036, 1.081, 1.044). This pattern suggests that uncorrelated Gaussian perturbations introduce instability into the training process without providing the structured exploration benefit that OU noise offers.

The competitive performance of the no-noise agent is explained by the 3000-step random warmup, which amounts to approximately 20 episodes of completely random actions. This warmup fills the replay buffer with diverse transitions before any learning begins. After warmup, the deterministic actor improves through gradient updates on this stored experience alone. The consistent (if slightly worse) performance of the no-noise agent shows that off-policy learning from the replay buffer can partially compensate for the lack of ongoing exploration.

The success rate metric highlights the difference between the methods more sharply than mean error. While all three achieve sub-1.1 mean error, OU noise reaches 10.5% success rate compared to 5.4% for Gaussian and 6.4% for no noise. This means the OU-trained agent more frequently brings the end-effector within 0.2 m of the target, suggesting that temporally correlated exploration specifically helps the policy learn fine positioning, not just coarse approach behavior.

The key finding of this study is that the structure of the noise matters more than its mere presence. Adding unstructured Gaussian noise provides essentially no benefit over having no noise at all, but adding temporally correlated OU noise yields approximately 7% improvement in tracking error and a roughly 2× improvement in success rate. For physical control systems with inertia and damping, the exploration noise must respect the temporal structure of the dynamics in order to be effective.

Chapter 7

Part (g): Generalization Tests (Target Distribution Shift)

Contents

7.1 Motivation **28**

7.2 Setup **28**

7.3 Results **29**

7.4 Discussion **30**

7.1 Motivation

A key question for any learned controller is whether it generalizes beyond its training distribution. A PID controller, being model-based and reactive, naturally handles any target trajectory within the workspace. An RL agent, in contrast, learns a policy optimized for the specific distribution of states encountered during training. This chapter investigates how well a DDPG agent trained on one trajectory type performs when evaluated on fundamentally different trajectories.

7.2 Setup

A DDPG agent is trained using Configuration B from Chapter 5 (hidden 256 $\rightarrow$ 128, 1000 episodes) on a circular motion trajectory (center $(0, 0)$, radius $r = 0.5$, angular velocity $\omega \approx 0.42\,\text{rad/s}$). It is then evaluated without further training on six test distributions that vary in speed, shape, size, and position, plus the original training distribution as a baseline:

Test Distribution	Description	Shift Type
Train (circular $\omega = 0.42$)	Same as training	None
Diff ω ($\omega = 1.0$)	Faster circular motion	Speed
Diff ω ($\omega = 2.0$)	Much faster circular motion	Speed
Diff $r = 1.0$	Larger radius circle	Scale
Diff center $(0.5, 0.5)$	Shifted circle center	Position
Lissajous	Figure-eight trajectory	Shape
Static $(1.5, 0)$	Fixed target point	Shape

Table 7.1: Test distributions for generalization evaluation.

7.3 Results

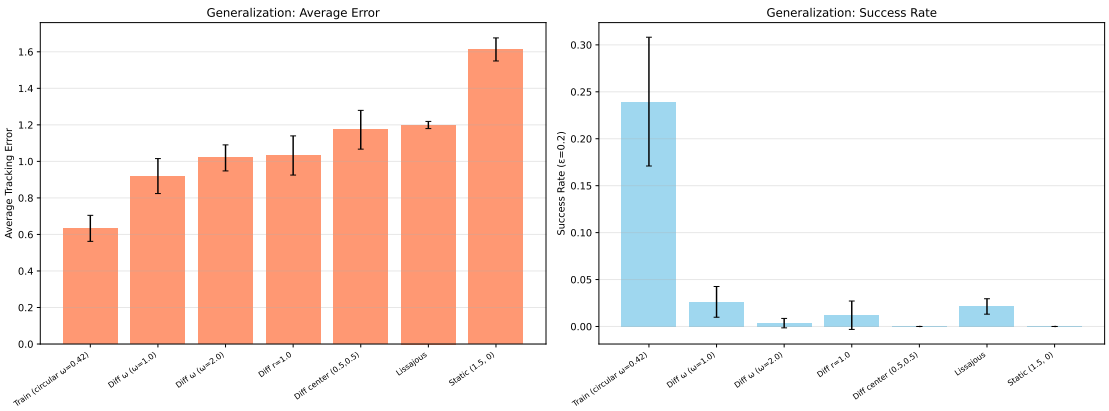

Figure 7.1: Generalization: average tracking error and success rate across test distributions (Configuration B, 1000 episodes).

Distribution	Avg Error	Success Rate	Return
Train (circular $\omega = 0.42$)	$\mathbf{0.633 \pm 0.071}$	$\mathbf{0.240 \pm 0.069}$	-102.6 ± 24.8
Diff $\omega = 1.0$	0.920 ± 0.096	0.026 ± 0.016	-168.3 ± 31.5
Diff $\omega = 2.0$	1.019 ± 0.071	0.004 ± 0.005	-196.2 ± 21.6
Diff $r = 1.0$	1.032 ± 0.107	0.012 ± 0.015	-206.4 ± 55.0
Diff center $(0.5, 0.5)$	1.173 ± 0.106	0.000 ± 0.000	-236.8 ± 38.3
Lissajous	1.199 ± 0.020	0.021 ± 0.008	-262.8 ± 14.0
Static $(1.5, 0)$	1.613 ± 0.063	0.000 ± 0.000	-452.6 ± 35.6

Table 7.2: Generalization performance across test distributions (mean ± std over 3 seeds).

7.4 Discussion

The 1000-episode agent achieves strong performance on its training distribution (0.633 error, 24% success rate), which is a substantial improvement over the 100-episode results reported in Chapter 5. However, a clear progression of degradation appears as the test conditions become more dissimilar from training.

Speed changes produce moderate degradation. Doubling the angular velocity to $\omega = 1.0$ increases the error from 0.633 to 0.920, and quadrupling it to $\omega = 2.0$ further increases it to 1.019. This monotonic decline with speed makes physical sense: a faster-moving target requires quicker reaction times and larger torques, and the agent was trained on a relatively slow trajectory. Nevertheless, the agent still tracks faster targets reasonably well, suggesting it has learned a general reaching behavior rather than memorizing the specific trajectory timing.

Changing the spatial characteristics of the trajectory is more challenging than changing its speed. Increasing the radius to $r = 1.0$ (1.032) and shifting the circle center to $(0.5, 0.5)$ (1.173) both push the target into workspace regions the agent rarely visited during training. The center-shifted circle is particularly difficult because the entire trajectory lies in a different part of the workspace, where the learned value function and policy may be poorly calibrated. The 0% success rate for the shifted center confirms that the agent cannot achieve fine tracking in these unfamiliar regions.

The Lissajous (figure-eight) trajectory produces an error of 1.199 with a non-zero success rate of 2.1%. Although the shape is fundamentally different from the training circle, the Lissajous pattern still involves smooth, continuous motion near the workspace

center. The agent can follow the general direction of motion but struggles with the more complex path reversals.

Static targets produce the worst performance by far (1.613). The agent was trained exclusively on moving targets and has learned a policy that expects continuous target motion. When the target is stationary at $(1.5, 0)$, the agent appears unable to converge and hold a fixed position, since this "reach and stop" behavior was never encountered during training. The high negative return (-452.6) confirms the agent accumulates large errors throughout the episode.

Overall, the results indicate that the DDPG agent has learned a general tracking behavior rather than memorizing the specific training trajectory, but its generalization is bounded by the diversity of states visited during training. Performance degrades gradually with speed changes, more sharply with spatial distribution shifts, and most severely when the task requires qualitatively different behavior (holding a static position). Training on a mixture of diverse target distributions would likely improve generalization across all these conditions.

Chapter 8

Part (h): Ablation Study on State Inputs

Contents

8.1 Motivation 32

8.2 State Representations 32

8.3 Results 33

8.4 Discussion 34

8.1 Motivation

The choice of state representation is a critical design decision in reinforcement learning. A good representation should contain all task-relevant information while remaining as compact as possible to facilitate efficient learning. For the reacher task, there are multiple natural ways to encode the state: raw observations, pre-computed errors, or combinations thereof. This ablation study investigates how different state representations affect DDPG's learning performance.

8.2 State Representations

Four state representations of varying dimensionality and information content are compared:

1. **Minimal (6-dim):** $s = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$

   The raw environment observation. Contains the target position and joint state, but the end-effector position must be inferred via forward kinematics.

2. **Minimal + End-effector (8-dim):** $s = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2, x_{\text{end}}, y_{\text{end}})$

Augments the minimal state with the computed end-effector position. This is the default representation used throughout the project.

3. **Error-only (6-dim):** $s = (e_x, e_y, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$ where $e_x = x_t - x_{\text{end}}$, $e_y = y_t - y_{\text{end}}$

   Replaces the separate target and end-effector positions with the Cartesian tracking error. This directly encodes what matters for the task but loses absolute position information.

4. **Error in IK space (4-dim):** $s = (\theta_1 - \theta_1^d, \theta_2 - \theta_2^d, \dot{\theta}_1, \dot{\theta}_2)$

   The most compact representation, expressing the error in joint space after inverse kinematics. Pre-computes the IK solution and provides only the joint-space error and velocities.

All four representations are trained with identical DDPG hyperparameters (Configuration B from Chapter 5: hidden 256 → 128, actor LR 2×10^{-4}, critic LR 6×10^{-4}; adjusted only for input dimension) for 1000 episodes with 3 seeds each.

8.3 Results

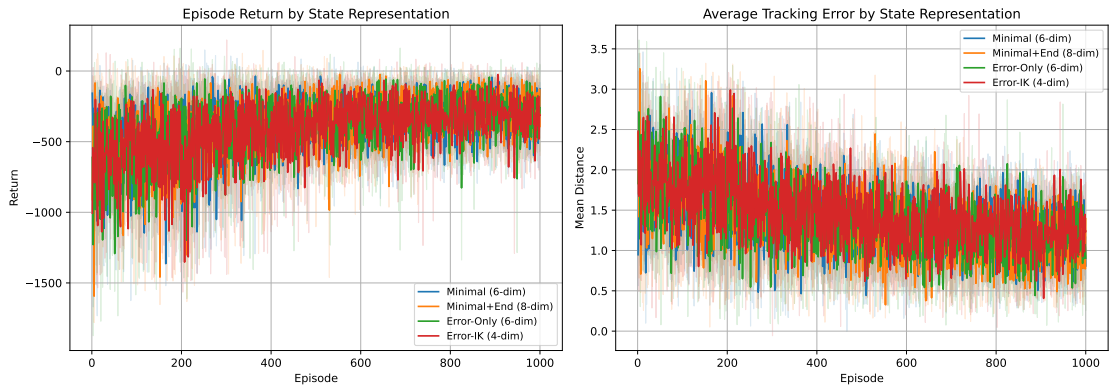

Figure 8.1: Ablation study: learning curves for different state representations (1000 episodes, Configuration B).

Representation	Dim	Avg Error (last 10)	Avg Return (last 10)
Error-Only	6	$\mathbf{1.048 \pm 0.551}$	-268.5 ± 236.0
Minimal	6	1.122 ± 0.437	-262.5 ± 182.9
Minimal + End-effector	8	1.124 ± 0.422	-277.1 ± 178.7
Error-IK	4	1.214 ± 0.373	-301.2 ± 176.4

Table 8.1: Ablation results: final performance by state representation (mean ± std over 3 seeds).

8.4 Discussion

With 1000 episodes of training, the top three representations (Error-Only, Minimal, Minimal+End) converge to very similar performance, all achieving mean errors near 1.1. Error-IK remains the clear outlier, roughly 16% worse than the best. Compared to the learning curves in the plot, all four representations show a slow warmup phase in the first ∼100 episodes (corresponding to the 3000-step random warmup), followed by gradual improvement. The top three largely overlap throughout training, while Error-IK consistently lags behind.

Error-Only (6-dim) achieves the lowest mean error at 1.048. By directly encoding the Cartesian tracking error (e_x, e_y), this representation provides the agent with the most task-relevant signal: the error vector immediately tells the agent how far and in which direction it needs to move. The 6-dimensional input also keeps the network compact, reducing the number of parameters the agent needs to fit. Interestingly, Error-Only also has the highest variance across seeds (±0.551), suggesting that while this representation enables the best average performance, it is more sensitive to the random initialization of the network and the exploration trajectory.

Minimal (6-dim) and Minimal+End (8-dim) achieve nearly identical errors (1.122 vs. 1.124). This is a notable result: adding the end-effector position as an explicit input provides essentially no benefit when the network has 1000 episodes to learn. The $256 \rightarrow 128$ network has sufficient capacity to learn the forward kinematics mapping internally, making the extra two dimensions redundant. Both representations also show lower variance than Error-Only, indicating more consistent learning across seeds.

Error-IK (4-dim) performs worst at 1.214. Although this is the most compact representation, the IK pre-computation discards too much information. The joint-space

error alone does not convey the absolute workspace position, so the critic cannot distinguish between states that have the same angular error but very different arm configurations (e.g., fully extended vs. folded). Additionally, IK singularities near the workspace boundary can produce discontinuous state transitions that disrupt learning. However, it is worth noting that even Error-IK achieves a reasonable error with only 4 input dimensions, demonstrating that DDPG can learn a useful policy from very compact state information.

The overall takeaway is that with sufficient training (1000 episodes), the DDPG agent is quite robust to the choice of state representation as long as the core task information is preserved. The top three representations are within 7% of each other, and the differences are smaller than the seed-to-seed variance within any single representation. The Error-Only encoding offers a slight edge by presenting the task-relevant information in the most direct form, while overly aggressive compression (Error-IK) incurs a measurable but not catastrophic penalty.

Chapter 9

Part (i): Metrics and Reporting Standards

Contents

9.1 Metrics Definition **36**

9.2 Evaluation Protocol **37**

9.3 Results **38**

9.4 Detailed Analysis **38**

9.4.1 Tracking Error and Success Rate 38

9.4.2 Control Energy 39

9.4.3 Smoothness 39

9.4.4 Key Trade-offs 40

9.1 Metrics Definition

To provide a comprehensive comparison across all control methods, we define four metrics that capture different aspects of controller performance:

1. **Mean tracking error:** $\bar{e} = \frac{1}{T}\sum_{t=1}^{T} \|p_{\text{end}}(t) - p_{\text{target}}(t)\|$

   The average Euclidean distance between the end-effector and the target over the episode. This is the primary performance metric.

2. **Success rate:** $\text{SR} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}(\|p_{\text{end}}(t) - p_{\text{target}}(t)\| < \varepsilon)$ with $\varepsilon = 0.2$ m

   The fraction of timesteps where the end-effector is within 0.2 m of the target. This metric is more stringent than mean error because it requires precise tracking, not just proximity.

3. **Control energy:** $E_\tau = \sum_{t=1}^{T} \|\tau_t\|^2$

The total squared torque magnitude over the episode. Lower energy indicates a more efficient controller, which matters for real robotic systems with limited battery life or motor thermal constraints.

4. **Smoothness:** $S_{\Delta\tau} = \sum_{t=2}^{T} \|\tau_t - \tau_{t-1}\|^2$

   The sum of squared torque differences between consecutive steps. Lower smoothness scores indicate more gradual torque changes, which reduce mechanical wear and produce less jerky motion. This metric penalizes bang-bang control strategies in particular.

9.2 Evaluation Protocol

To ensure a fair comparison, all methods are evaluated on the same set of starting conditions. Each method uses a fixed set of random seeds (seed + 1000 for seeds 42, 123, 456), so that PID controllers and RL agents face identical target trajectories and initial arm configurations. RL methods (SARSA and DDPG) are each trained for 100 episodes before evaluation. DDPG uses the lightweight Configuration A from Chapter 5 (hidden 64 $\rightarrow$ 32, aggressive learning rates). Results are reported as mean $\pm$ std across $K = 3$ seeds.

9.3 Results

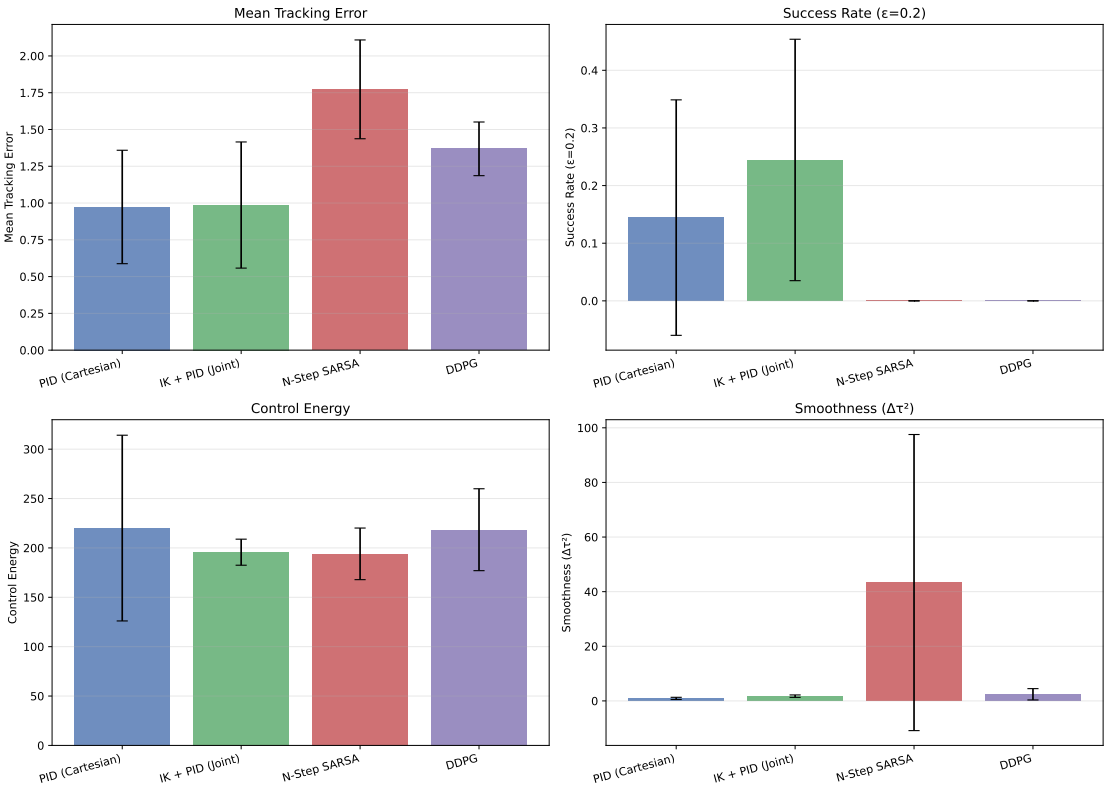

Figure 9.1: Final metrics comparison across all methods.

Method	Mean Error	Success Rate	Ctrl Energy	Smoothness
PID (Cartesian)	0.973 ± 0.386	0.144 ± 0.204	220.1 ± 94.0	0.9 ± 0.4
IK + PID (Joint)	0.987 ± 0.429	0.244 ± 0.209	195.7 ± 13.2	1.7 ± 0.4
N-Step SARSA	1.773 ± 0.336	0.000 ± 0.000	194.0 ± 26.1	43.3 ± 54.2
DDPG	1.369 ± 0.182	0.000 ± 0.000	218.4 ± 41.5	2.4 ± 2.1

Table 9.1: Final metrics comparison across all methods (mean ± std over 3 seeds).

9.4 Detailed Analysis

9.4.1 Tracking Error and Success Rate

The PID controllers achieve the lowest mean tracking errors (Cartesian: 0.973, IK+PID: 0.987), leveraging their knowledge of the system dynamics through the Jacobian transpose and inverse kinematics respectively. Despite similar mean errors,

IK+PID achieves a notably higher success rate (24.4%) compared to Cartesian PID (14.4%), because joint-space control produces more consistent behavior across different arm configurations.

Both RL methods have zero success rate, meaning the end-effector never comes within 0.2 m of the target at any timestep. This highlights the gap between learned and model-based controllers under limited training: with 100 episodes, the RL policies can reduce the overall distance to the target but cannot achieve the fine positioning required for "success." Among the RL methods, DDPG (1.369) substantially outperforms SARSA (1.773), demonstrating the clear advantage of continuous action spaces.

9.4.2 Control Energy

SARSA achieves the **lowest control energy** (194.0), slightly below IK+PID (195.7). This may seem surprising, but it reflects the fact that SARSA's discrete action set $\{-1, 0, +1\}^2$ includes the zero-torque option $(0, 0)$, and an undertrained agent often selects it, applying no torque at all. DDPG (218.4) and Cartesian PID (220.1) use comparable energy, as both produce continuous torques driven by error signals.

9.4.3 Smoothness

The smoothness metric reveals stark contrasts between methods. The PID controllers achieve the best smoothness (Cartesian: 0.9, IK+PID: 1.7), which is expected since their torque outputs change continuously based on the smooth error signal. DDPG achieves a smoothness of 2.4, which is slightly worse than PID but orders of magnitude better than SARSA (43.3).

SARSA's poor smoothness is a direct consequence of its discrete action space. The agent can only choose from 9 torque values $\{-1, 0, +1\}^2$, so every action change causes an abrupt torque discontinuity. DDPG, on the other hand, produces continuous outputs that allow for gradual torque transitions and much smoother control. The roughly 18× smoothness advantage of DDPG over SARSA underscores that continuous control is essential for this class of problems.

9.4.4 Key Trade-offs

Model knowledge vs. data efficiency. Classical controllers exploit system dynamics knowledge to achieve immediate good performance. RL methods must learn the dynamics from interaction alone, and require many episodes to approach comparable accuracy.

Tracking precision vs. energy efficiency. PID controllers prioritize tracking accuracy by applying whatever torque is needed, while SARSA and DDPG discover policies that balance tracking with energy conservation through the reward signal.

Discrete vs. continuous actions. The SARSA/DDPG comparison isolates the effect of action space design. DDPG's continuous actions outperform SARSA in tracking error (1.37 vs. 1.77) and smoothness (2.4 vs. 43.3), though both achieve zero success rate under the 100-episode training budget.

Data budget matters. With only 100 episodes (15,000 transitions), the RL methods are operating in a severely data-limited regime. As shown in Chapter 5, extending DDPG training to 1,000 episodes with a larger network reduces the mean error from 1.43 to 1.15, suggesting that with sufficient training, DDPG could further close the gap with PID on tracking precision.

Chapter 10

Conclusion

This project implemented and compared four control approaches for a two-link planar reacher, spanning the spectrum from classical model-based control to model-free deep reinforcement learning.

10.1 Summary of Methods

1. **Cartesian PD Control** (Chapter 2): A reactive controller operating in task space via the Jacobian transpose. Simple and effective for known dynamics, achieving a mean tracking error of 0.973. The gain tuning process revealed that integral action is counterproductive for short-horizon tasks (3 seconds), leading to a pure PD design with $K_p = 10$, $K_d = 8$.

2. **IK + Joint PD Control** (Chapter 3): Separates the problem into inverse kinematics and joint-space tracking. Achieves the highest success rate (24.4%) among all methods due to more consistent behavior across arm configurations. Uses PD gains of $K_p = 20$, $K_d = 10$ per joint.

3. **N-Step SARSA** (Chapter 4): A discrete RL approach limited by the coarse 9-action space. Despite extensive hyperparameter tuning (adjusting N from 8 to 4, tripling the learning rate, using aggressive epsilon decay), the discrete formulation produces poor smoothness (43.3) and cannot match continuous controllers. An experiment with 25 actions confirmed that the bottleneck is the discrete paradigm itself, not the action resolution.

4. **DDPG** (Chapter 5): A continuous RL approach investigated under three configurations. The lightweight Configuration A (100 episodes, 64 $\rightarrow$ 32) achieves a tracking error of 1.43, Configuration B (1000 episodes, 256 $\rightarrow$ 128) reduces this to 1.15, and Configuration C (1000 episodes with deferred learning, ELU activations, and burst updates) achieves the best result of 0.90. All produce smooth

torque profiles (smoothness 2.4 for Configuration A), far superior to SARSA's 43.3. Since Configuration C relies on non-standard techniques beyond the scope of this project, Configuration B was used as the base for all subsequent experiments.

10.2 Key Findings

1. **Model knowledge enables precision; learning discovers different strategies.** Classical PD controllers achieve lower tracking error than DDPG (0.97 vs. 1.37 for 100-episode DDPG), which is expected since they have access to the system dynamics. However, with 1000 episodes of training, DDPG reduces its error to 1.15, closing about 55% of the gap. The trade-off between precision and training budget is fundamental: model-based controllers compute reasonable actions immediately, while learned policies require extensive interaction to reach comparable levels.

2. **100 episodes is a severe constraint for RL.** Both SARSA and DDPG are operating far from convergence. DDPG's off-policy replay buffer helps it learn more efficiently (each transition is reused multiple times), but 100 episodes provide only 15,000 transitions, far less than the millions typically used in continuous control benchmarks. The tuning process focused heavily on extracting maximum learning from minimal data through aggressive exploration schedules, early warmup termination, and fast learning rates.

3. **Continuous actions are essential for physical control.** The SARSA vs. DDPG comparison is a controlled experiment on action space design. DDPG outperforms SARSA in tracking error (1.37 vs. 1.77) and smoothness (2.4 vs. 43.3). For torque-controlled robots, discrete actions produce physically unacceptable bang-bang behavior.

4. **Temporal correlation in exploration noise matters more than noise presence** (Chapter 6). OU noise (0.985 error, 10.5% success rate) outperforms both Gaussian noise (1.057, 5.4%) and no noise (1.054, 6.4%) when trained for 1000 episodes. Notably, unstructured Gaussian noise provides essentially no benefit over having no noise at all, while temporally correlated OU noise yields a $\sim$7% error reduction and roughly doubles the success rate. This indicates that the

structure of the exploration, not merely its presence, is what matters for physical control.

5. **DDPG is robust to state representation with sufficient training** (Chapter 8). With 1000 episodes, the top three representations (Error-Only, Minimal, Minimal+End) converge to nearly identical errors (1.05 to 1.12), within 7% of each other and smaller than the seed-to-seed variance. Error-Only (6-dim) performs best by directly encoding the tracking error, while the most compact representation (Error-IK, 4-dim) is 16% worse due to discarding workspace context through IK pre-computation. The key lesson is that task-relevant information matters more than dimensionality, and overly aggressive compression hurts more than redundant features.

6. **Generalization degrades gradually with distribution shift** (Chapter 7). The 1000-episode DDPG agent achieves 0.633 error on its training distribution (circular, $\omega = 0.42$) but degrades to 0.920 for moderate speed changes ($\omega = 1.0$), 1.173 for spatial shifts (different circle center), and 1.613 for static targets. The degradation follows a clear pattern: speed changes are tolerated best, spatial shifts cause moderate degradation, and qualitatively different behaviors (holding a position) are handled worst. Training on a diverse mixture of target distributions would likely improve generalization across all conditions.

10.3 Lessons Learned from Tuning

The hyperparameter tuning process throughout this project yielded several practical lessons:

- **Match network size to training budget.** A compact $64 \rightarrow 32$ network performed best with 100 episodes, while $256 \rightarrow 128$ was needed to benefit from 1000 episodes. Larger networks on small budgets simply cannot fill their parameter space with useful information.

- **Integral action is harmful for short episodes.** Both PID controllers performed better without K_i. In the 3-second window, the integrator accumulates transient error and causes overshoot rather than improving steady-state accuracy.

- **Exploration must end early with limited budgets.** With 100 episodes, epsilon decay of 0.90 (reaching ϵ_{min} by episode 28) outperformed 0.97 (reaching ϵ_{min} by episode 98). Similarly, DDPG noise decay of 0.93 outperformed slower schedules. The agent needs most of its episodes for exploitation and refinement.

- **Reward scaling interacts with learning rate.** The raw quadratic reward caused training instabilities in the 100-episode DDPG configuration. A scale factor of 0.5 provided the best balance for that regime, while the slower learning rates in the 1000-episode configuration could handle the full reward scale.

- **Failed experiments are informative.** The 25-action SARSA experiment confirmed that the bottleneck is the discrete paradigm, not the action resolution. The large-network DDPG experiment on the 100-episode budget showed that more parameters do not help when data is scarce. These negative results guided subsequent design choices.